

# VLAN hopping

Follow-along: play the attacker, then the defender

You have three hosts on two switches. Two VLANs keep them apart: VLAN 10 on 103.10.0.0/24, VLAN 20 on 103.20.0.0/24.

| Role     | IP          | VLAN | Switch | Container           |
|----------|-------------|------|--------|---------------------|
| attacker | 103.10.0.10 | 10   | S1     | 103_L2_LAB_attacker |
| peer     | 103.10.0.11 | 10   | S2     | 103_L2_LAB_peer     |
| victim   | 103.20.0.20 | 20   | S2     | 103_L2_LAB_victim   |

The two switches (103\_L2\_LAB\_S1 and 103\_L2\_LAB\_S2) are joined by one 802.1Q trunk that carries both VLANs. The switch interface inside each host is named after its switch: 103-S1 on the attacker, 103-S2 on the peer and the victim. There is no router: VLAN 10 and VLAN 20 are separate Layer-2 domains with no path between them. That separation is what you are about to break.

The attacker carries a stock Layer-2 kit; there is no bespoke exploit to run:

- **ip** (iproute2): reads the interfaces and, in step 3, adds an 802.1Q subinterface for the single-tag attempt.
- **scapy**: builds the double-tagged frame one tag at a time, so you set the outer and inner VLAN ids yourself.
- **tcpdump**: run on the victim, it is the oracle; it shows whether your frame crossed into VLAN 20.

A one-command tool, **vlan-double-tag**, is also on the attacker as a baseline. This walkthrough ignores it and builds the frame by hand in scapy so you set each tag and see why two are needed. Open a shell wherever a step says so with:

```
docker exec -it <container> bash
```

**Assessment.** This lab is graded on fourteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all fourteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step; none needs anything beyond the lab in front of you.

## Part 1: the attack

1. **Confirm the honest baseline.** On the attacker, reach the two other hosts and watch which one answers:

```
docker exec -it 103_L2_LAB_attacker bash
ping -c2 103.10.0.11          # the peer: VLAN 10, other switch -> reachable
ping -c2 -W1 103.20.0.20     # the victim: VLAN 20 -> "Network is unreachable"
```

The peer answers: it shares VLAN 10 with you, and the trunk carries VLAN 10 between the switches, so the two of you are one broadcast domain across both switches. The victim does not: it sits in VLAN 20, a different subnet with no route from yours.

**Question 1.** The attacker reaches the peer on the far switch but not the victim on the same far switch. Both are one hop across the same trunk. What separates the victim from you that the peer does not have, and why does the trunk carrying both VLANs not already join them?

2. **Read the switch's VLAN plan.** The hop exploits how the ports are configured, so read them first. On S1:

```
docker exec -it 103_L2_LAB_S1 bash
ovs-vsctl get port 103-attacker vlan_mode tag trunks    # native-untagged, 10,
[10]
ovs-vsctl get port 103-S2          vlan_mode tag trunks    # native-untagged, 10,
[10, 20]
```

Two facts matter. Your access port 103-attacker is **native-untagged** with **tag=10**: it accepts your untagged traffic as VLAN 10, and it also accepts a frame already tagged with VLAN 10. The trunk 103-S2 has **tag=10** as well; on a trunk that **tag** is the *native VLAN*, the one VLAN sent across the trunk without a tag. The native VLAN here is 10, the same as your access VLAN.

**Question 2.** On the trunk port 103-S2, **tag=10** sets the native VLAN. State what "native VLAN" means for a frame leaving a trunk: for VLAN 10 versus VLAN 20, which one goes onto the trunk with an 802.1Q tag and which goes without?

3. **Try the simple way first, and watch it fail.** The obvious move is to tag your traffic as VLAN 20 directly. Add a VLAN 20 subinterface and use it. Start a capture on the victim in one shell:

```
docker exec -it 103_L2_LAB_victim bash
tcpdump -n -i 103-S2 icmp          # leave running; watch for anything from you
```

Then on the attacker:

```
docker exec -it 103_L2_LAB_attacker bash
ip link add link 103-S1 name 103-S1.20 type vlan id 20
ip addr add 103.20.0.66/24 dev 103-S1.20
ip link set 103-S1.20 up
ping -c3 -W1 103.20.0.20          # tagged VLAN 20: dropped at your own port
ip link del 103-S1.20             # remove it before step 4
```

The victim's **tcpdump** stays silent. Your port 103-attacker trunks only VLAN 10 (**trunks=[10]**), so a frame that arrives tagged VLAN 20 is dropped the moment it ingresses; it never reaches the trunk.

**Question 3.** Your single-tagged VLAN 20 frame is dropped at 103-attacker before it goes anywhere. Which part of that port's config rejects it, and what would have to be true of the port for a plain single VLAN 20 tag to reach the victim?

4. **Double-tag the frame.** The trick is to wrap the VLAN 20 tag inside a VLAN 10 tag. The outer VLAN 10 tag gets your frame accepted at your port and then stripped when the frame leaves the trunk in the native VLAN; the inner VLAN 20 tag survives that strip

and is read fresh by S2. Build it by hand in scapy (keep the victim's `tcpdump` from step 3 running):

```
docker exec -it 103_L2_LAB_attacker bash
scapy
>>> sendp(Ether(dst="ff:ff:ff:ff:ff:ff")/Dot1Q(vlan=10)/Dot1Q(vlan=20)
...      /IP(src="103.10.0.10", dst="103.20.0.20")/ICMP(),
...      iface="103-S1", count=5)
```

Each field, and why:

- `Dot1Q(vlan=10)`: the outer tag. It equals the trunk's native VLAN, so it is the tag that gets stripped on the way out of the trunk.
- `Dot1Q(vlan=20)`: the inner tag, the VLAN you are hopping into. It is payload to S1 and a real tag to S2.
- `Ether(dst=ff:ff:ff:ff:ff:ff)`: broadcast, so you do not need the victim's MAC and cannot be asked for one across the VLAN boundary. Every host in VLAN 20 receives it; the victim acts on it because the inner IP names its address.
- `count=5`: five frames, enough to see them land.

The one-command equivalent is `vlan-double-tag 103-S1 10 20 103.20.0.20 5`; it takes the same four values so you still name both VLANs and the target.

**Question 4.** Name where each of the two tags is removed: which switch strips the outer VLAN 10 tag, at what moment, and which switch reads the inner VLAN 20 tag. Why does a single switch never strip both tags in one pass?

**Question 5.** The outer tag must be VLAN 10, not any other id. Tie this to step 2: what is the exact relationship between the outer tag and the trunk's native VLAN that makes the strip happen, and what would the frame do if you set the outer tag to VLAN 99 instead?

5. **See the hop.** On the victim's `tcpdump` you now see ICMP echo requests arriving, sourced from 103.10.0.10: a VLAN 10 address, the attacker's own, showing up inside VLAN 20. Your frame crossed a boundary that has no router and no route. That is the confirmed attack.

**Question 6.** The victim's capture shows the request but the attacker never gets a reply. Trace the victim's reply: it is a normal VLAN 20 frame addressed back to 103.10.0.10. Explain why it does not return to the attacker, and what that makes double-tagging good for and bad for.

**Question 7.** You addressed the frame to the broadcast MAC rather than the victim's MAC. Why can the attacker not simply learn the victim's MAC first the way it would for a host in its own VLAN, and what does using broadcast cost the attacker in precision?

6. **Confirm the separation was real.** So there is no doubt the hop, not a misconfiguration, carried the frame: on the attacker, check you still have no route to VLAN 20, and that the peer (VLAN 10) is who you can reach normally:

```
ip route                                # only 103.10.0.0/24; nothing toward 103.20.0.0/24
ping -c1 103.10.0.11                    # peer, VLAN 10: still your only honest reach
```

**Question 8.** At L3 the attacker has no route to 103.20.0.0/24, yet the frame reached a VLAN 20 host. Explain why VLAN hopping is a Layer-2 attack that a Layer-3 control (a firewall rule between the subnets, say) would not have stopped.

## Part 2: the defence

Your job on the defender side is to make the hop fail while the attacker keeps its honest VLAN 10 reach to the peer. Two fixes; each closes a different precondition, and either one stops the attack. Apply them on the switches, not on the hosts.

### Option A: strict access ports

The hop starts because your port 103-attacker accepts a tagged frame at all. An access port should carry one VLAN and refuse tagged ingress outright. In Open vSwitch that is `vlan_mode=access`: it accepts only untagged frames and drops any 802.1Q-tagged frame on ingress, whatever VLAN the tag names. Set it on S1:

```
docker exec -it 103_L2_LAB_S1 bash
ovs-vsctl set port 103-attacker vlan_mode=access tag=10
ovs-vsctl clear port 103-attacker trunks
ovs-vsctl get port 103-attacker vlan_mode tag trunks    # access, 10, []
```

Re-run the attack from step 4. The victim's `tcpdump` stays empty: the double-tagged frame carries an outer VLAN 10 tag, and a strict access port drops it on ingress before the outer tag is ever stripped. Confirm the honest path survives: `ping -c2 103.10.0.11` from the attacker still works, because your ordinary traffic is untagged and an access port accepts untagged frames.

Reference solution: `solution/access_ports.sh` (locks the attacker, peer, and victim ports to strict access).

**Question 9.** A strict access port drops the double-tagged frame on ingress. The peer's port was already `vlan_mode=access` in the baseline. Explain why the attack worked anyway, i.e. why fixing only the attacker's port matters and the already-strict peer and victim ports were not the hole.

**Question 10.** After this fix the attacker's untagged VLAN 10 traffic to the peer still passes, but every tagged frame from the attacker is dropped. Why does refusing tagged ingress cost the attacker its hop without costing an ordinary VLAN 10 host anything?

### Option B: move the native VLAN off the attacker's VLAN

The strip in step 4 happens only because the trunk's native VLAN (10) equals your access VLAN (10). Break that equality: park the native VLAN on an unused id that no host uses, on both ends of the trunk. First put the ports back to the vulnerable baseline (`scripts/reset.sh`), then:

```
docker exec -it 103_L2_LAB_S1 bash
ovs-vsctl set port 103-S2 tag=999    # S1's trunk end: native VLAN now 999
exit
docker exec -it 103_L2_LAB_S2 bash
ovs-vsctl set port 103-S1 tag=999    # S2's trunk end: match it
```

Re-run the attack. Now VLAN 10 is no longer the native VLAN, so it leaves the trunk *tagged*. Your outer VLAN 10 tag is not stripped; it stays on the frame all the way to S2, which reads the frame as VLAN 10 and delivers it to VLAN 10 ports, never to the victim. The inner VLAN 20 tag stays buried in the payload and is never read. The peer (VLAN 10) is still reachable, because VLAN 10 still crosses the trunk; it just crosses tagged now.

Reference solution: `solution/native_vlan.sh`.

**Question 11.** With the native VLAN moved to 999, the outer VLAN 10 tag is no longer stripped at the trunk. Walk the double-tagged frame from the attacker to S2 under this config and say exactly where it ends up and why the victim never sees it.

**Question 12.** The classic hardening advice is "never leave the native VLAN at the default VLAN 1." This lab's native VLAN was 10, not 1, and moving it to 999 still fixed the hop. State the general rule this is an instance of: what must the native VLAN never be equal to, and why?

After either fix, the victim's `tcpdump` stays empty while the attacker fires the double-tagged frame, and `ping 103.10.0.11` from the attacker still works. The hop is stopped and the honest VLAN 10 path is intact.

**Question 13.** Option A refuses the tagged frame at ingress; Option B stops the tag from being stripped on the trunk. A network runs strict access ports on every access port but leaves a trunk native VLAN equal to a live access VLAN. Is it still vulnerable to this attack? Answer for both the case where the attacker sits on a strict access port and the case where the attacker can reach a trunk directly.

## Using the hop when no reply comes back

Step 5 showed the hop is one-way: your frame reaches VLAN 20, but the victim's reply stays in VLAN 20 and never returns to you. That looks like it should make the attack useless. It does not. Three kinds of follow-up work with a send-only primitive.

**Acting without a reply.** Some protocols do their whole job on one datagram and answer nothing you need. An SNMP SET with a guessed community string reconfigures a switch, printer, or power distribution unit; a forged syslog line drops false evidence into a VLAN 20 log collector; many building-automation and industrial UDP protocols take a command and act on it. You never read the response because the attack does not depend on one.

**Poisoning a whole segment.** A double-tagged frame sent to the broadcast MAC reaches every host in VLAN 20 at once. A forged gratuitous ARP that claims the gateway's IP repoints those hosts at a MAC that does not exist in their VLAN, so their off-subnet traffic falls into a hole. You cannot intercept that traffic, which would need a reply path; you can deny it, a targeted blackhole on a segment your routing cannot reach.

**Borrowing a reply path.** Most of the time the hop is only the way in, and the goal is a two-way channel. Deliver an exploit to a service listening on UDP in VLAN 20 and have the payload connect back out to you: the compromised host, not the double tag, carries the return traffic, over its own normal routing. Or write something a host in VLAN 20 acts on later and read the result through a path you already reach. The one-way limit stops mattering once something inside the VLAN answers on your behalf.

The same lax port has more uses than double-tagging. A port that accepts a tagged frame also accepts spanning-tree and trunk-negotiation frames, and a port that can be talked into becoming a full trunk hands the attacker every VLAN, two-way, which beats one-way injection

outright. That is why Part 2 locks the port down instead of filtering one attack: the fixes cut the chain at its first link.

**Question 14.** Every follow-up above is either connectionless or arranges its own way back; none opens an ordinary TCP session across the hop. Explain what stops the attacker from completing a TCP handshake to a VLAN 20 service from its send-only position, and give the general test this yields for whether any attack can work over a one-way hop.

## Checks: what "done" means

- **Attack works:** with the vulnerable baseline in place, the victim's `tcpdump -n -i 103-S2 icmp` shows the attacker's double-tagged frames arriving from 103.10.0.10, while a normal `ping` from the attacker to 103.20.0.20 still fails.
- **Defence holds:** with either fix in place, the same attack leaves the victim's capture empty, and the attacker's `ping 103.10.0.11` to the peer still succeeds.

`scripts/status.sh` prints each switch's port VLAN modes, the attacker-to-peer reach, and the hop oracle (it fires the attack and counts what the victim receives), so you can read the before and after without shelling in. `scripts/reset.sh` stops the attack and returns the ports to the Part-1 baseline; `scripts/teardown.sh` removes the lab.

**Safety.** Everything here lives on two isolated Open vSwitch bridges with no route off the host. The double-tagged frame only ever reaches the lab's own containers.